



Chapter - 2

Elementary Programming

Writing a Simple Java Program

1. What is an Algorithm? (The Planning Stage)

Before you write any code, you need a plan.

- **Algorithm:** A step-by-step list of actions needed to solve a problem. Think of it as a recipe.
- **Pseudocode:** Writing the algorithm in a mix of normal English and simple code to make it easy to understand.
- **Pro Tip:** Always outline your program as an algorithm before typing any actual Java code.

Example Algorithm (To find the area of a circle):

1. Read the circle's radius.
2. Compute the area using the formula: $\text{area} = \text{radius} * \text{radius} * 3.14159$
3. Display the result.

2. The Basic Structure of a Java Program

Every Java program needs a specific "skeleton" to run.

- **The Class:** Every program starts with a class definition.
 - Example: `public class ComputeArea { }`
- **The Main Method:** This is the starting point of your program. The computer looks for this exact line to begin executing your code:
 - Example: `public static void main(String[] args) { }`

3. Variables & Data Types (Storing Information)

To calculate the area, the computer needs to remember the radius. It does this using variables.

- **Variable:** A named container that stores a value in the computer's memory.
 - Every variable has a name, a data type, a size, and a value.
 - Use descriptive names (use `radius` and `area`, do not use `x` and `y`).
- **Data Types:** You must tell Java what kind of data the variable will hold (like whole numbers, text, or decimals). These basic types are called *primitive data types*.
- **Floating-Point Numbers:** This is computer-speak for "numbers with a decimal point."
 - To store a decimal number in Java, use the keyword `double`.
 - Example: `double radius;` (This tells Java: "Create a container named 'radius' that will hold a decimal number.")

Here is how the algorithm translates into an actual, working Java program:

```
ComputeArea.java

1 public class ComputeArea {
2     public static void main(String[] args) {
3         double radius; // Declare radius
4         double area; // Declare area
5
6         // 2. Assign a value to the radius
7         radius = 20;
8
9         // 3. Compute the area
10        area = radius * radius * 3.14159;
11
12        // 4. Display the results
13        System.out.println("The area for the circle of radius " + radius + " is " + area);
14    }
15 }
```

4. Tracing a Program (How to Debug)



Tracing means reviewing your program line-by-line to see how the variables change in memory. It is a great tool for understanding your code and finding errors.

line#	radius	area
3	no value	
4		no value
7	20	
10		1256.636

5. String Concatenation (The + Sign)

In Java, the + symbol does two different things:

1. **Addition:** Adds two numbers together (e.g., 5 + 5).
2. **String Concatenation:** Joins text (strings) and numbers together into one long sentence.
 - In the code above, "The area is " + area takes the text part, converts the calculated number into text, and glues them together for the final output.

Reading Input from the Console

1. Why Read Input?

- **The Problem:** In the previous example, the radius was "hardcoded" (radius = 20;). To calculate a different radius, you would have to change the actual code and recompile the program.
- **The Solution:** Reading input from the console allows the user to type in their own values while the program is running.

2. Standard Input and Output

- **System.out:** Refers to the standard output device (your display monitor). We use it to print things to the screen.
- **System.in:** Refers to the standard input device (your keyboard). We use it to receive data from the user.

3. The Scanner Class (Your Input Tool)

Since Java does not support console input directly out of the box, you must use a special tool called the Scanner class.

- **Creating a Scanner:** You must write this exact line of code to create a Scanner object that listens to the keyboard:

```
Scanner input = new Scanner(System.in);
```
- **Reading a Number:** Once the Scanner is created, you can tell it to wait for the user to type a decimal number by using the `nextDouble()` method: `double radius = input.nextDouble();`

4. Prompting the User (print vs. println)

Before asking the user to type something, you should always display a message (a prompt) so they know what to do.

- **println("..."):** Prints the text and moves the cursor to the next line.
- **print("..."):** Prints the text but keeps the cursor on the same line. (This is usually better for input prompts so the user types right next to the question).

5. The import Statement



The Scanner class does not load automatically. It lives in a package called `java.util`. You must tell Java to bring it into your program by putting an import statement at the very top of your file.

- **Specific Import:** Imports only the Scanner class. `import java.util.Scanner;`
- **Wildcard Import:** Imports all classes in the `java.util` package using an asterisk (*). `import java.util.*;`
- **Note:** There is no performance difference between the two. Use whichever you prefer.

Code Example 1: Compute Area with User Input

```
ComputeAreaWithConsoleInput.java

1 import java.util.Scanner; // Step 1: Import the Scanner
2
3 public class ComputeAreaWithConsoleInput {
4     public static void main(String[] args) {
5         // Step 2: Create a Scanner object
6         Scanner input = new Scanner(System.in);
7
8         // Step 3: Prompt the user
9         System.out.print("Enter a number for radius: ");
10
11        // Step 4: Read the input from the keyboard
12        double radius = input.nextDouble();
13
14        // Step 5: Compute and display the result
15        double area = radius * radius * 3.14159;
16        System.out.println("The area for the circle of radius " + radius + " is " + area);
17    }
18 }
```

6. Reading Multiple Inputs

You can use the Scanner to read more than one number at a time. The user can type the numbers separated by spaces or by pressing the Enter key after each one.

- **Warning:** If the program expects a number and the user types letters instead, the program will crash (this is called a runtime error).

Code Example 2: Computing an Average

```
ComputeAreaWithConsoleInput.java

1 import java.util.Scanner;
2
3 public class ComputeAverage {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         // Prompt for three numbers
8         System.out.print("Enter three numbers: ");
9
10        // The program waits for the user to type three decimal numbers
11        double number1 = input.nextDouble();
12        double number2 = input.nextDouble();
13        double number3 = input.nextDouble();
14
15        // Compute the average
16        double average = (number1 + number2 + number3) / 3;
17
18        // Display results
19        System.out.println("The average of " + number1 + " " + number2 + " " + number3 + " is " +
20        average);
21    }
22 }
```

Identifiers

Definition: Identifiers are simply the names you create to identify elements in your program, such as classes, methods, and variables (e.g., `main`, `input`, `number1`, `ComputeArea`).



The Rules of Naming: Java is very strict about how you name things. All identifiers must follow these exact rules:

1. **Allowed Characters:** They can only contain letters, numbers (digits), underscores (`_`), and dollar signs (`$`).
2. **Starting Character:** They must start with a letter, an underscore (`_`), or a dollar sign (`$`). They cannot start with a number.
3. **No Reserved Words:** You cannot use words that Java has already claimed for its own code (like `public`, `class`, or `import`).
4. **Special Values:** You cannot use the words `true`, `false`, or `null`.
5. **Length:** They can be of any length.

Examples of Valid Identifiers: `$2`, `ComputeArea`, `area`, `radius`, `print`

Examples of Invalid Identifiers:

- `2A` (starts with a number)
- `d+4` (contains a `+` sign, which is not allowed)

Note: If you break these rules, the Java compiler will report a syntax error and your program will not run.

Variables

Definition: Variables are containers used to store data in the computer's memory. They are called "variables" because their values can vary (change) while the program is running.

Reassigning Values: You can update the value of a variable at any point in your code.

- **Example:** If you set `radius = 1.0;`, you can change it later by writing `radius = 2.0;`. The computer will forget `1.0` and store `2.0`.

Before you can use a variable, you must declare it. This tells the compiler the variable's name and what data type it will hold, which allows the computer to set aside the right amount of memory.

- **Standard Declaration:** `datatype variableName;`

`int count; // Declares a variable that holds whole numbers`

`double interestRate; // Declares a variable that holds decimal numbers`

Multiple Declarations: If multiple variables share the exact same data type, you can declare them together on one line, separated by commas.

`int i, j, k; // Declares three integer variables at once`

Initializing Variables

Variables often need a starting value. This is called initialization.

- **Two-Step Method:** Declare first, then assign a value

```
int count;
```

```
count = 1;
```

One-Step Method (Recommended): You can declare and initialize a variable at the exact same time.

```
int count = 1;
```

- **Multiple Initializations:** You can also declare and initialize multiple variables of the same type on one line.

```
int i = 1, j = 2;
```

Variable Scope

Definition: A variable's "scope" refers to the specific part of the program where that variable is allowed to be used or referenced.



The Golden Rule: A variable must be declared and initialized before you can use it in any calculations or output statements.

Assignment Statements & Assignment expressions

After you declare a variable, you use an assignment statement to give it a value. In Java, the equal sign (=) is the assignment operator.

- **The Formula:** `variable = expression;`
- **What is an Expression?** An expression is a mix of values, variables, and math operators that calculate down to a single result.
 - Example: `int x = 5 * (3 / 2);` (The math part on the right is the expression).
- **The Golden Rule:** The variable name **MUST** always go on the left side of the equal sign.
 - Correct: `x = 1;`
 - Wrong: `1 = x;` (The computer cannot assign the value of x to the number 1).

Updating a Variable using Itself: You can use the same variable on both sides of the equal sign to update its value.

`x = x + 1;` // Takes the current value of x, adds 1, and saves it back into x.

Multiple Assignments at Once: You can assign the same value to several variables at once.

`i = j = k = 1;` // Sets k to 1, then j to 1, then i to 1

Named Constants

What is a Constant? While variables can change, a constant represents permanent data that remains the same throughout the program (like the mathematical value of Pi: 3.14159).

How to Declare: You must declare and initialize a constant in the exact same line using the keyword `final`.

1. **Syntax:** `final datatype CONSTANTNAME = value;`
2. **Example:** `final double PI = 3.14159;`

3 Benefits of Constants:

1. **Saves Time:** You don't have to repeatedly type long numbers.
2. **Easy to Update:** If you need to change the value (e.g., making PI 3.14159265), you only change it in one place, not everywhere in the code.
3. **Readability:** `radius * radius * PI` is much easier for a human to understand than `radius * radius * 3.14159`.

Code Example:



```
ComputeAreaWithConstant.java

1 import java.util.Scanner;
2
3 public class ComputeAreaWithConstant {
4     public static void main(String[] args) {
5         // Step 1: Declare a constant for PI
6         final double PI = 3.14159;
7
8         // Step 2: Create a Scanner object
9         Scanner input = new Scanner(System.in);
10
11         // Step 3: Prompt the user to enter a radius
12         System.out.print("Enter a number for radius: ");
13         double radius = input.nextDouble();
14
15         // Step 4: Compute area using the constant PI
16         double area = radius * radius * PI;
17
18         // Step 5: Display result
19         System.out.println("The area for the circle of radius " + radius + " is " + area);
20     }
21 }
```

Naming Conventions

Java is case-sensitive (meaning Area and area are treated as two completely different names). Following standard naming rules prevents bugs and makes your code look professional.

- **Rule 1: Variables and Methods (camelCase)**
 - Start with a lowercase letter. If the name has multiple words, capitalize the first letter of every subsequent word.
 - Examples: radius, area, print, computeArea, interestRate
- **Rule 2: Classes (PascalCase)**
 - Capitalize the first letter of every word in the name.
 - Examples: ComputeArea, System, Scanner
- **Rule 3: Constants (ALL_CAPS)**
 - Capitalize every single letter. Separate multiple words using an underscore (_).
 - Examples: PI, MAX_VALUE

Numeric data type and operations

1. Numeric Data Types (Storing Numbers)

Every data type has a specific range. Java allocates memory space based on the type you choose.

Integer Types (Whole Numbers): There are four types (short, byte, int, long) of integers. You should choose the one that fits your data size, but int is used most of the time for simplicity.

Floating-Point Types (Decimals): There are two types (float, decimal) for numbers with decimal points.

**TABLE 2.1** Numeric Data Types

Name	Range	Storage Size
byte	-2^7 to $2^7 - 1$ (−128 to 127)	8-bit signed
short	-2^{15} to $2^{15} - 1$ (−32768 to 32767)	16-bit signed
int	-2^{31} to $2^{31} - 1$ (−2147483648 to 2147483647)	32-bit signed
long	-2^{63} to $2^{63} - 1$ (i.e., −9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: $-3.4028235E + 38$ to $-1.4E - 45$ Positive range: $1.4E - 45$ to $3.4028235E + 38$	32-bit IEEE 754
double	Negative range: $-1.7976931348623157E + 308$ to $-4.9E - 324$ Positive range: $4.9E - 324$ to $1.7976931348623157E + 308$	64-bit IEEE 754

- Always prefer double because it is twice as big and much more accurate than float.

2. Reading Inputs from the Keyboard

Just like `nextDouble()` reads a decimal number, the Scanner class has specific methods to read every other numeric data type.

Method	Description
<code>nextByte()</code>	reads an integer of the byte type.
<code>nextShort()</code>	reads an integer of the short type.
<code>nextInt()</code>	reads an integer of the int type.
<code>nextLong()</code>	reads an integer of the long type.
<code>nextFloat()</code>	reads a number of the float type.
<code>nextDouble()</code>	reads a number of the double type

Warning: If the user types a number that is too big for the type (e.g., typing 128 into a byte), the program will crash with a runtime error.

3. Numeric Operators

Java uses standard math symbols for calculations. The numbers being operated on are called operands.

TABLE 2.3 Numeric Operators

Name	Meaning	Example	Result
+	Addition	$34 + 1$	35
−	Subtraction	$34.0 - 0.1$	33.9
*	Multiplication	$300 * 30$	9000
/	Division	$1.0 / 2.0$	0.5
%	Remainder	$20 \% 3$	2

The Remainder Operator (%)

The % operator, also known as the remainder or modulo operator, gives you the remainder left over after division.

- The number on the left is the dividend, and the number on the right is the divisor.
- **Examples:** 7 % 3 yields 1 (3 goes into 7 twice, with 1 left over).
 - 12 % 4 yields 0 (4 goes into 12 perfectly).
 - 26 % 8 yields 2.

Negative Numbers and Modulo: You can also use % with negative integers and floating-point values. The rule is simple: The remainder is negative ONLY if the dividend (the left number) is negative.

- -7 % 3 yields -1
- -12 % 4 yields 0
- -26 % -8 yields -2
- 20 % -13 yields 7

Why is the Remainder Operator Useful? It is incredibly useful in programming for solving real-world logic problems:

1. **Finding Even or Odd Numbers:** Any even number % 2 is always 0, and any odd number % 2 is always 1.
2. **Calculating Days:** If today is Saturday (Day 6), what day is it in 10 days?
 - (6 + 10) % 7 is 2. (Day 2 of the week is Tuesday!).

Code Example: DisplayTime.java

This program takes a large amount of seconds and uses both division (/) and modulo (%) to figure out the minutes and the leftover seconds.

```
DisplayTime.java

1 import java.util.Scanner;
2
3 public class DisplayTime {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6         // Prompt the user for input
7         System.out.print("Enter an integer for seconds: ");
8         int seconds = input.nextInt();
9
10        int minutes = seconds / 60; // Find minutes in seconds
11        int remainingSeconds = seconds % 60; // Seconds remaining
12        System.out.println(seconds + " seconds is " + minutes + " minutes and " + remainingSeconds + "
seconds");
13    }
14 }
```

4. Unary vs. Binary Operators

- **Unary Operator:** Operates on a single number. (e.g., The - in -5 simply makes the 5 negative).
- **Binary Operator:** Operates on two numbers. (e.g., The - in 4 - 5 performs subtraction).

5. Exponent Operations

To calculate exponents (like a^b), you cannot use standard symbols like ^ in Java. Instead, you must use a built-in tool called `Math.pow(a, b)`.

- **Syntax:** `Math.pow(base, exponent)`
- **Note:** This method always returns a double (floating-point number) as its result.



```
1 System.out.println(Math.pow(2, 3)); // 2 cubed. Displays 8.0
2 System.out.println(Math.pow(4, 0.5)); // Square root of 4. Displays 2.0
3 System.out.println(Math.pow(2.5, 2)); // 2.5 squared. Displays 6.25
```

Numeric Literals

1. What is a Literal?

A literal is a constant value that you type directly into your source code.

- **Examples:** In the code `int numberOfYears = 34;`, the number 34 is the literal.

2. Integer Literals (Whole Numbers)

When you type a whole number into Java, it automatically assumes it is an `int` type.

- **The Size Limit:** You can assign an integer literal to any integer variable (`byte`, `short`, `int`), but it must fit within that variable's size limit.
 - **Example:** `byte b = 128;` causes a compile error because the maximum size for a `byte` is 127.
- **The long Literal:** If you need to write a massive number that is larger than the maximum `int` value (over 2 billion), you must add an `L` (or lowercase `l`) to the very end of the number to tell Java it is a long literal.
 - **Best Practice:** Always use the uppercase `L` (e.g., 2147483648L) because a lowercase `l` looks too much like the number 1.

Different Number Systems: By default, Java assumes numbers are decimal (Base 10). However, you can write literals in other systems using specific prefixes:

- **Binary (Base 2):** Use `0b` or `0B` (e.g., `0B1111`)
- **Octal (Base 8):** Use a leading `0` (e.g., `07777`)
- **Hexadecimal (Base 16):** Use `0x` or `0X` (e.g., `0xFFFF`)

3. Floating-Point Literals (Decimals)

When you type a number with a decimal point (like 5.0), Java automatically assumes it is a `double` type.

- **Making it a float:** If you want to store a decimal strictly as a smaller `float` type, you must append an `f` or `F` to the end of the literal.
 - Example: `100.2F` or `100.2f`
- **Making it explicitly a double:** You can optionally append a `d` or `D`, though it isn't required since it is the default.
 - Example: `100.2D` or `100.2d`

Why double is better than float: The `double` type is significantly more accurate because it stores more digits.

- `float` has about 7 to 8 significant digits.
- `double` has about 15 to 17 significant digits.

```
// Example of precision differences:
System.out.println(1.0 / 3.0); // Double: prints 0.3333333333333333 (16 digits)
System.out.println(1.0F / 3.0F); // Float: prints 0.33333334 (8 digits)
```

4. Scientific Notation

For extremely large or microscopic decimal numbers, you can write floating-point literals using scientific notation.

- Instead of writing "times 10 to the power of", Java uses the letter `E` (or `e`).
- **Formula:** $a \times 10^b$ becomes `aEb`
- **Examples:** 1.23456×10^2 is written as `1.23456E2` or `1.23456E+2`



- 1.23456×10^{-2} is written as `1.23456E-2`

Fun Fact: Floating-point numbers are called "floating" because internally, the computer converts them into scientific notation, moving (or "floating") the decimal point to a new position.

5. Readability Trick: Underscores

If you are typing a long literal (like a credit card number or a phone number), you can insert underscores (`_`) between the digits to make the code easier for humans to read. The Java compiler simply ignores them.

- **Correct Usage:** `long creditCard = 2324_4545_4519_3415L;`
- **The Rule:** The underscore must be sandwiched between two numbers. You cannot put it at the very beginning or the very end of a number (`_45` and `45_` will cause errors).

Evaluating Expressions and Operator Precedence

Java evaluates expressions exactly the same way you would solve standard arithmetic problems in math class. However, you must explicitly write out every operator. (For example, in math you can write $4x$, but in Java, you must write `4 * x`).

- **Example Translation:**
 - **Math Formula:** $(3+4x-x)/5-10(y-5)(a+b+c)+9(x+4+y+9x)/y$
 - **Java Code:** `(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x + 9 * (4 / x + (9 + x) / y)`

1. Operator Precedence (Order of Operations)

When an expression has multiple operators, Java follows a strict order of evaluation (just like PEMDAS/BODMAS in math):

1. **Parentheses () First:** Everything inside parentheses is calculated first. If parentheses are nested inside each other, the innermost ones are evaluated first.
2. **Multiplication *, Division /, and Remainder % Second:** If there are multiple of these in a row, Java calculates them from left to right.
3. **Addition + and Subtraction - Last:** If there are multiple of these, they are also calculated from left to right.

2. Step-by-Step Evaluation Example

Here is exactly how Java's brain processes a complex expression step-by-step:

`3 + 4 * 4 + 5 * (4 + 3) - 1`

- **Step 1 (Parentheses):** Calculates $(4 + 3)$. The expression becomes `3 + 4 * 4 + 5 * 7 - 1`
- **Step 2 (Multiplication, Left):** Calculates $4 * 4$. The expression becomes `3 + 16 + 5 * 7 - 1`
- **Step 3 (Multiplication, Right):** Calculates $5 * 7$. The expression becomes `3 + 16 + 35 - 1`
- **Step 4 (Addition, Left):** Calculates $3 + 16$. The expression becomes `19 + 35 - 1`
- **Step 5 (Addition, Right):** Calculates $19 + 35$. The expression becomes `54 - 1`
- **Step 6 (Subtraction):** Calculates $54 - 1$ to get the final answer: 53

3. The Integer Division Trap

Be incredibly careful when using division in formulas! Because dividing two integers in Java chops off the decimal, it can ruin your calculations.

- **Example:** If you want to use the fraction $\frac{95}{9}$ in a formula, writing `5 / 9` will yield 0.
- **The Fix:** Make at least one number a decimal, like `5.0 / 9`, which correctly yields 0.55555...

Code Example: Fahrenheit to Celsius

This program converts Fahrenheit to Celsius using the formula

`Celsius = 95 * (Fahrenheit - 32)`



```
FahrenheitToCelsius.java

1 import java.util.Scanner;
2
3 public class FahrenheitToCelsius {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         // 1. Prompt the user for Fahrenheit
8         System.out.print("Enter a degree in Fahrenheit: ");
9         double fahrenheit = input.nextDouble();
10
11        // 2. Convert Fahrenheit to Celsius (Notice the 5.0!)
12        double celsius = (5.0 / 9) * (fahrenheit - 32);
13
14        // 3. Display the result
15        System.out.println("Fahrenheit " + fahrenheit + " is " + celsius + " in Celsius");
16    }
17 }
```

Case Study: Display the current time

This case study is highly important because it demonstrates how to use programming logic (specifically division and modulo) to solve a real-world problem. Let's break this down deeply!

1. The Main Goal

We need to write a program that fetches the current time from the computer and displays it in GMT (Greenwich Mean Time) using the format Hour:Minute:Second (e.g., 13:19:8).

2. The Magic Tool & The UNIX Epoch

Computers do not natively store time in the standard HH:MM:SS format.

- **System.currentTimeMillis():** This is a built-in Java method. When you call it, it returns a massive number representing the total milliseconds.
- **The UNIX Epoch (Time Zero):** When did time "start" for computers? Midnight on January 1, 1970, GMT. This is known as the "UNIX Epoch."
- Therefore, System.currentTimeMillis() tells you exactly how many milliseconds have passed from Jan 1, 1970, up to this exact second (e.g., 1203183068328 milliseconds).

3. The Deep Logic: / (Division) vs % (Remainder)

We only have a massive chunk of total milliseconds. We need to extract the current time from it using / (to find total chunks of time) and % (to find the current position on the clock face). Understanding the difference between Total and Current is crucial here:

- **Step 1: Milliseconds to Seconds**
 - Divide total milliseconds by 1000 (/ 1000) because there are 1000 ms in 1 second.
 - Result: totalSeconds (All the seconds that have passed since 1970).
- **Step 2: Find Current Second**
 - The seconds hand on a clock only goes from 0 to 59. If we take totalSeconds and find the remainder when divided by 60 (% 60), we get the current second.
 - Formula: currentSecond = totalSeconds % 60;
- **Step 3: Seconds to Minutes**
 - Divide totalSeconds by 60 (/ 60).
 - Result: totalMinutes (All the minutes that have passed since 1970).
- **Step 4: Find Current Minute**
 - The minute hand also only goes from 0 to 59.
 - Formula: currentMinute = totalMinutes % 60;
- **Step 5: Minutes to Hours**



- Divide `totalMinutes` by 60 (`/ 60`).
- Result: `totalHours` (All the hours that have passed since 1970).
- **Step 6: Find Current Hour**
 - A day has 24 hours (0 to 23). To find the current hour, find the remainder when divided by 24.
 - Formula: `currentHour = totalHours % 24;`

4. The Code:

Notice how the logic above translates directly into code. (Also notice that the data type `long` is used because the number of milliseconds is way too large for a standard `int`).

```
ShowCurrentTime.java

1 public class ShowCurrentTime {
2     public static void main(String[] args) {
3
4         // Step 1: Obtain the total milliseconds since midnight, Jan 1, 1970
5         long totalMilliseconds = System.currentTimeMillis();
6
7         // Step 2 & 3: Obtain total seconds and compute the current second
8         long totalSeconds = totalMilliseconds / 1000;
9         long currentSecond = totalSeconds % 60;
10
11        // Step 4 & 5: Obtain total minutes and compute the current minute
12        long totalMinutes = totalSeconds / 60;
13        long currentMinute = totalMinutes % 60;
14
15        // Step 6 & 7: Obtain total hours and compute the current hour
16        long totalHours = totalMinutes / 60;
17        long currentHour = totalHours % 24;
18
19        // Step 8: Display the final result
20        System.out.println("Current time is " + currentHour + ":"
21                           + currentMinute + ":" + currentSecond + " GMT");
22    }
23 }
```

5. Important Points to Remember

- **Why long?** Total milliseconds is a massive number (in the billions). It will not fit in a standard `int` memory space, so using the `long` data type is mandatory to prevent a compile error.
- **The Formatting Issue:** If the current second is 8, the output will be `13:19:8`. In the real world, we want it to display as `13:19:08`. You will learn how to format this single digit with a leading 0 in later chapters (using String formatting).
- **Timezones:** This code defaults to `GMT` (Greenwich Mean Time). If you want your local time, you would need to add or subtract offsets to the `currentHour` and `currentMinute`.

Augmented Assignment Operator

1. What is an Augmented Assignment Operator?

In programming, you will very often need to take the current value of a variable, do some math with it, and then save the new result right back into that exact same variable.

- **Standard Way:** `count = count + 1;`



- **The Shortcut:** Java lets you combine the math and the assignment into one step using an augmented (or compound) assignment operator.
- **Shortcut Way:** `count += 1;`

2. The Five Augmented Operators

Here are all the shorthand operators you can use. Notice how the math symbol always comes before the equal sign.

Operator	Name	Example	What the computer actually reads
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>

3. Crucial Rules & Warnings

Rule 1: The Right Side is Evaluated First When you use an augmented operator with a complex math problem, the computer will solve the entire right side of the equation first before applying the augmented operator.

- Example: `x /= 4 + 5.5 * 1.5;`
- What Java does: `x = x / (4 + 5.5 * 1.5);`
- **Warning:** It does NOT do `x = (x / 4) + ...`

Rule 2: No Spaces Allowed You cannot put a space between the math symbol and the equal sign.

- Correct: `+=`
- Wrong (Compile Error): `+=`

Rule 3: Statements vs. Expressions Just like the standard `=` sign, these operators can be used as standalone instructions (statements) or inside other commands (expressions).

```
x += 2;           // Used as a standard statement
System.out.println(x += 2); // Used as an expression (adds 2, then prints the result immediately)
```

Increment And Decrement Operators

1. What are Increment and Decrement Operators?

In programming, you will constantly need to increase or decrease a variable's value by exactly

1. Java provides two very handy shorthand operators for this:

- **Increment Operator (++):** Adds 1 to the variable. (Pronounced "plus plus")
- **Decrement Operator (--):** Subtracts 1 from the variable. (Pronounced "minus minus")

Example:

```
int i = 3, j = 3;
i++; // i becomes 4 (same as i = i + 1)
j--; // j becomes 2 (same as j = j - 1)
```



Operator	Name	Description	Example (assume i= 1)
++var	preincrement	Increment var by 1, and use the new var value in the statement	<code>int j = ++i; // j is 2, i is 2</code>
var++	postincrement	Increment var by 1, but use the original var value in the statement	<code>int j = i++; // j is 1, i is 2</code>
--var	predecrement	Decrement var by 1, and use the new var value in the statement	<code>int j = --i; // j is 0, i is 0</code>
var--	postdecrement	Decrement var by 1, and use the original var value in the statement	<code>int j = i--; // j is 1, i is 0</code>

2. Prefix vs. Postfix (The Placement Matters!)

You can place these operators either before or after the variable. If the variable is standing completely alone on its own line, both do the exact same thing. But if they are used inside a larger math equation, they act very differently!

- **Postfix (After the variable):** `i++` or `i--`
 - Rule: USE the current value first, THEN UPDATE it.
- **Prefix (Before the variable):** `++i` or `--i`
 - Rule: UPDATE the value first, THEN USE the new value.

3. The Big Difference: Step-by-Step Examples

Let's look at how the computer reads these differently when doing math:

Example 1: Using Postfix (`i++`)

```
int i = 10;
int newNum = 10 * i++;
```

How Java reads it:

1. It sees `i++`, so it uses the old value of `i` (10) for the math.
2. `newNum = 10 * 10;` (`newNum` becomes 100).
3. After the math is done, `i` is incremented. `i` becomes 11.

Example 2: Using Prefix (`++i`)

```
int i = 10;
int newNum = 10 * (++i);
```

How Java reads it:

1. It sees `++i`, so it updates `i` immediately. `i` becomes 11.
2. Now it uses the new value for the math.
3. `newNum = 10 * 11;` (`newNum` becomes 110).

4. Complex Example:

```
double x = 1.0;
double y = 5.0;
double z = x-- + (++y);
```

- **Step 1 (`++y`):** Prefix! `y` goes up by 1 immediately. `y` becomes 6.0.
- **Step 2 (The Math):** Java uses the updated `y` (6.0), but the old `x` (1.0) because `x` has a postfix.
 - `z = 1.0 + 6.0;`
 - `z` becomes 7.0.
- **Step 3 (`x--`):** Postfix! Now that the math is totally done, `x` goes down by 1. `x` becomes 0.0.

5. Crucial Programming Tip

While `++` and `--` make your code shorter, they can make equations incredibly confusing to read.



- **Best Practice:** Avoid using these operators inside complex expressions (like `int k = ++i + i;`).
- Keep it simple! It is usually much better to increment your variables on their own separate lines before doing the math.

Numeric Type Conversion (Casting)

1. Automatic Conversion in Math

If you mix an integer and a floating-point number in a math problem, Java automatically converts the integer into a floating-point value before doing the math.

- Example: `3 * 4.5` is automatically treated by Java as `3.0 * 4.5`.

2. Widening vs. Narrowing Types

Every data type has a size limit. Converting between them is called type casting.

- **Widening (Automatic):** Moving a value from a small data type to a larger one (like an `int` to a `long`). Java does this safely and automatically.
- **Narrowing (Manual):** Moving a value from a large data type to a smaller one (like a `double` to an `int`). You must do this manually using explicit casting, otherwise, the program will crash with a compile error.

3. How to Cast (Syntax)

To narrow a type manually, you must put the target data type inside parentheses right in front of the value or variable you want to change.

- **The Formula:** `(targetType) value`
- **Example:** `System.out.println((int)1.7);`

4. Crucial Rules for Casting

- **Decimals are completely chopped off:** When casting a decimal (`double` or `float`) to an integer (`int`), Java truncates the fractional part. It does not round. `(int)1.7` becomes `1`.
- **Variables do not permanently change:** Casting only changes the value for that specific line of code. The original variable remains totally unchanged in memory.
 - Example: If `double d = 4.5;` and you run `int i = (int)d;`, `i` becomes `4`, but `d` stays `4.5`.
- **Fixing Integer Division:** As learned earlier, `1 / 2` yields `0`. But if you cast one number first, like `(double)1 / 2`, it turns into `1.0 / 2`, which correctly yields `0.5`.

5. The Augmented Assignment

Augmented operators (like `+=`) have a hidden feature: they automatically perform explicit casting behind the scenes.

- If you have `int sum = 0;` and you run `sum += 4.5;`, you will not get an error.
- Java automatically reads it as `sum = (int)(sum + 4.5);`. The value of `sum` simply becomes `4`.

6. Variables vs. Literals Exception

- Assigning an `int` literal (like the number `10`) to a `byte` variable works automatically without casting, as long as the number fits in the `byte` size limit.
- However, assigning an `int` variable to a `byte` variable will always cause an error unless you explicitly cast it, even if the stored number is small.

Code Example: Sales Tax



```
SalesTax.java

1 import java.util.Scanner;
2
3 public class SalesTax {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         System.out.print("Enter purchase amount: ");
8         double purchaseAmount = input.nextDouble();
9
10        // Let's say purchaseAmount is 197.55
11        double tax = purchaseAmount * 0.06; // tax becomes 11.853
12
13        // The Formatting Trick:
14        // 1. Multiply by 100: 11.853 * 100 = 1185.3
15        // 2. Cast to (int) to chop off the decimal: (int)1185.3 = 1185
16        // 3. Divide by 100.0 to float the decimal back: 1185 / 100.0 = 11.85
17
18        System.out.println("Sales tax is $" + (int)(tax * 100) / 100.0);
19    }
20 }
```

Software Development Process

1. The Software Development Life Cycle (SDLC)

Developing software is an engineering process. No matter the size of the project, it generally follows a continuous life cycle. If errors occur at any stage, developers step back to a previous stage to fix them.

- **Requirements Specification:** Meeting with the customer to deeply understand the problem and formally document exactly what the software needs to do.
- **System Analysis:** Identifying the data flow. You determine the desired Output first, and then figure out what Input is required to produce it.
- **System Design:** Designing the step-by-step process (the algorithm) to convert the inputs into the outputs. This breaks the problem down into smaller, manageable subsystems.
- **Implementation (Coding):** Translating the design into actual programming code (like Java). This includes writing the code, self-testing, and debugging (fixing errors).
- **Testing:** An independent team verifies that the code meets all initial requirements and is free of bugs.
- **Deployment:** Making the software available to the actual users (e.g., installing it on computers or hosting it on the internet).
- **Maintenance:** Continuously updating the software to fix newly discovered bugs and add improvements in a changing environment.

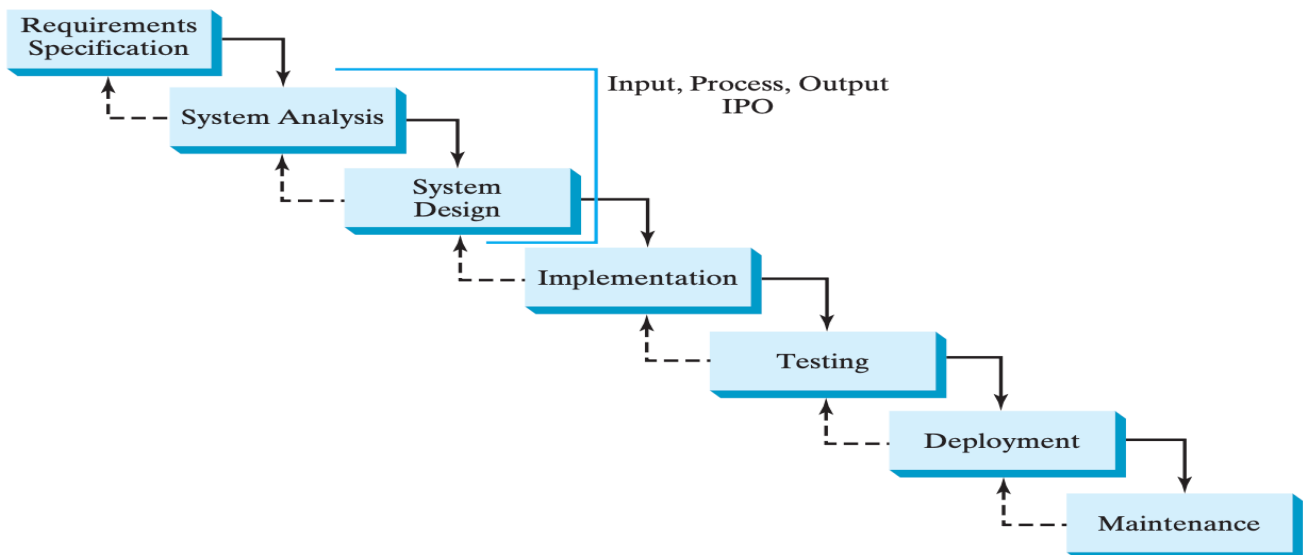


FIGURE 2.3 At any stage of the software development life cycle, it may be necessary to go back to a previous stage to correct errors or deal with other issues that might prevent the software from functioning as expected.

2. Case Study: Computing a Loan Payment

To see the SDLC in action, we can build a program to calculate loan payments (IPO: Input, Process, Output).

Stage 1: Requirements Specification

- **Goal:** The program must let the user enter the interest rate, the loan amount, and the number of years.
- **Output Expected:** It must compute and display the monthly payment and the total payment.

Stage 2: System Analysis (The Math)

The outputs can be found using standard mathematical formulas.

- **Note:** As a programmer, you don't always need to know how to invent these formulas; you just need to know how to translate them into code!

$$\text{monthlyPayment} = \frac{\text{loanAmount} \times \text{monthlyInterestRate}}{1 - \frac{1}{(1 + \text{monthlyInterestRate})^{\text{numberOfYears} \times 12}}}$$

$$\text{totalPayment} = \text{monthlyPayment} \times \text{numberOfYears} \times 12$$

Stage 3: System Design (The 5-Step Logic)

During design, we outline the exact steps the computer will take:

- **Step 1:** Prompt the user to enter the annual interest rate, the number of years, and the loan amount.
- **Step 2:** Convert the annual interest rate into a monthly decimal. (Because users enter a percentage like 4.5%, you must divide it by 100 to get a decimal, and then divide by 12 for the months. Shortcut: Divide the annual rate by 1200).
- **Step 3:** Compute the monthly payment using the formula.
- **Step 4:** Compute the total payment (monthly payment $\times$ 12 $\times$ number of years).
- **Step 5:** Display the final monthly payment and total payment.



Stage 4: Implementation (The Code) Here is the code translation of our design. Notice how the `Math.pow()` method is used to calculate the exponent from the formula, and how casting `(int)` is used at the very end to format the money to exactly two decimal places!

```

ComputeLoan.java

1 import java.util.Scanner;
2
3 public class ComputeLoan {
4     public static void main(String[] args) {
5         // Create a Scanner
6         Scanner input = new Scanner(System.in);
7
8         // 1. Get Inputs
9         System.out.print("Enter annual interest rate, e.g., 7.25%: ");
10        double annualInterestRate = input.nextDouble();
11
12        System.out.print("Enter number of years as an integer, e.g., 5: ");
13        int numberOfYears = input.nextInt();
14
15        System.out.print("Enter loan amount, e.g., 120000.95: ");
16        double loanAmount = input.nextDouble();
17
18        // 2. Process (Math)
19        // Convert annual rate to a monthly decimal
20        double monthlyInterestRate = annualInterestRate / 1200;
21
22        // Calculate the monthly payment
23        double monthlyPayment = loanAmount * monthlyInterestRate / (1
24        - 1 / Math.pow(1 + monthlyInterestRate, numberOfYears * 12));
25
26        // Calculate the total payment
27        double totalPayment = monthlyPayment * numberOfYears * 12;
28
29        // 3. Output (Using the casting trick to format to 2 decimal places)
30        System.out.println("The monthly payment is $" +
31        (int)(monthlyPayment * 100) / 100.0);
32
33        System.out.println("The total payment is $" +
34        (int)(totalPayment * 100) / 100.0);
35    }
36 }

```

Stage 5: Testing

Once the code is written, you must test it!

- Run the program and enter sample data (e.g., \$120,000 for 5 years at 7.25%).
- Grab a calculator and verify if the program's output matches the real mathematical answer.
- **Pro Tip:** When writing your code, build and test it incrementally (one step at a time) rather than writing the whole file at once. This makes it much easier to find bugs!

Case Study: Counting Monetary Units

This case study is a classic programming problem. It teaches you how to combine Type Casting, Integer Division (`/`), and the Remainder/Modulo Operator (`%`) to build a real-world tool like a cash register.

1. The Main Goal

We want to write a program that takes a total amount of money (like \$11.56) and breaks it down into the minimum number of coins/bills.



- The order of units is: Dollars (100 cents), Quarters (25 cents), Dimes (10 cents), Nickels (5 cents), and Pennies (1 cent).

2. The Golden Rule of Money in Java

Never do this math with decimals (double)! Dealing with decimals in division and modulo can get very messy.

- **The Trick:** Convert the entire dollar amount into cents first.
- **Example:** Instead of dealing with \$11.56, multiply it by 100 and cast it to an int to get 1156 cents. Now you are only working with whole numbers!

3. The Deep Logic: / (How Many?) vs % (What is Left?)

Just like the Time case study, we will use / to find out how many coins fit, and % to find out how much money is left over for the next coin. Let's trace the amount 11.56:

- **Step 1:** Convert to Cents
 - $11.56 * 100 = 1156$ total cents
- **Step 2:** Find Dollars (100 cents)
 - How many? $1156 / 100 = 11$ Dollars
 - What is left? $1156 \% 100 = 56$ cents remaining
- **Step 3:** Find Quarters (25 cents)
 - How many? $56 / 25 = 2$ Quarters (which is 50 cents)
 - What is left? $56 \% 25 = 6$ cents remaining
- **Step 4:** Find Dimes (10 cents)
 - How many? $6 / 10 = 0$ Dimes
 - What is left? $6 \% 10 = 6$ cents remaining
- **Step 5:** Find Nickels (5 cents)
 - How many? $6 / 5 = 1$ Nickel
 - What is left? $6 \% 5 = 1$ cent remaining
- **Step 6:** Find Pennies (1 cent)
 - The remaining amount is simply the pennies. 1 Penny.

4. The Code: ComputeChange.java

Notice how the variable `remainingAmount` is constantly updated using the % operator after every single coin calculation!



ComputeChange.java

```
1 import java.util.Scanner;
2
3 public class ComputeChange {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         // Step 1: Receive the amount as a decimal
8         System.out.print("Enter an amount in double, for example 11.56: ");
9         double amount = input.nextDouble();
10
11         // Step 2: Convert to total cents (Notice the casting to int!)
12         int remainingAmount = (int)(amount * 100);
13
14         // Step 3: Find Dollars
15         int numberOfOneDollars = remainingAmount / 100;
16         remainingAmount = remainingAmount % 100; // Update remaining amount
17
18         // Step 4: Find Quarters
19         int numberOfQuarters = remainingAmount / 25;
20         remainingAmount = remainingAmount % 25;
21
22         // Step 5: Find Dimes
23         int numberOfDimes = remainingAmount / 10;
24         remainingAmount = remainingAmount % 10;
25
26         // Step 6: Find Nickels
27         int numberOfNickels = remainingAmount / 5;
28         remainingAmount = remainingAmount % 5;
29
30         // Step 7: Find Pennies
31         int numberOfPennies = remainingAmount;
32
33         // Step 8: Display results
34         System.out.println("Your amount " + amount + " consists of");
35         System.out.println("    " + numberOfOneDollars + " dollars");
36         System.out.println("    " + numberOfQuarters + " quarters ");
37         System.out.println("    " + numberOfDimes + " dimes");
38         System.out.println("    " + numberOfNickels + " nickels");
39         System.out.println("    " + numberOfPennies + " pennies");
40     }
41 }
```

5. The Floating-Point Precision Bug

There is a massive flaw in this program!

Because floating-point numbers (double) are stored in scientific notation internally, they are sometimes slightly inaccurate.

- **The Problem:** If you enter 10.03, the computer calculates $10.03 * 100$ as 1002.9999999999999.
- When you cast it to an (int), it chops off the decimal entirely. So 1002.999... becomes exactly 1002 cents.
- Your program will output 10 Dollars and 2 Pennies instead of 3 Pennies!
- **The Fix:** In real-world banking apps, programmers ask users to enter the amount directly as whole cents (an int), or they use special rounding methods to prevent this data loss.



To help you visualize exactly how / and % break down a total amount, you can play around with this interactive change calculator! Try entering 11.56 or your own numbers.

Common Errors and Pitfalls

1. Error (Breaking a Rule)

- **What is it?** You broke a strict rule of the Java language (a syntax or logic error).
- **Does the code run?** No. It will either refuse to compile or crash while running.
- **Do you get a warning?** Yes. The Java compiler gives you a direct error message and points to the exact line.
- **Example:** Trying to use a variable that you haven't declared yet.

2. Pitfall (A Trap or Bad Practice)

- **What is it?** A logical trap. Your code is technically "legal" in Java, but it is highly inefficient or produces the wrong answer.
- **Does the code run?** Yes. It will run 100% smoothly without crashing.
- **Do you get a warning?** No. The computer thinks everything is fine. You have to find the problem yourself.
- **Example:** Creating a brand new Scanner input = new Scanner(System.in) every single time you need an input, which wastes computer memory

Elementary programming errors often relate to how variables are managed and how mathematics is performed by the computer. Here are the most common mistakes to watch out for.

Common Error 1: Variable Issues (Undeclared, Uninitialized, Unused)

- **The Rule:** A variable must be declared (given a type) and initialized (given a value) before you try to use it.
- **Case Sensitivity Error:** Java is strictly case-sensitive.
 - If you declare `double interestRate = 0.05;` but try to use `interestrates * 45`, the program will crash because it thinks `interestrates` is a completely new, undeclared variable.
- **Unused Variables:** If you declare a variable (e.g., `double taxRate = 0.05;`) but never use it anywhere in your calculations or output, it is inefficient. Most modern IDEs (like Eclipse or NetBeans) will give you a warning to delete it.

Common Error 2: Integer Overflow

Every integer type has a strict size limit. An int can store numbers up to 2,147,483,647.

- **What is Overflow?** If you try to add 1 to the maximum limit, the computer's memory "rolls over" to the lowest possible negative number (−2,147,483,648).
- **The Danger:** Java does not give you a warning or an error when overflow happens. Your calculations will simply become wildly incorrect. Always be mindful when working with massive numbers.

(Note: If a floating-point number becomes too microscopically small for the computer to store, it is called underflow. Java simply approximates this to 0.0, which is usually harmless.)

Common Error 3: Round-off Errors



Because computers have limited memory space, they cannot store infinite decimals (like the true value of $1/3$). Floating-point numbers (double and float) are stored internally as scientific approximations.

- **The Result:** Doing math with floating-point numbers can lead to microscopic inaccuracies.

```
System.out.println(1.0 - 0.9); // Displays 0.09999999999999998, NOT exactly 0.1
```

- **The Rule:** Integers are precise; floating-point numbers are approximations. Keep this in mind when comparing decimal values or dealing with currency.

Common Error 4: Unintended Integer Division

Java uses the / symbol for both integer division and decimal division.

- **The Trap:** If both numbers are integers, Java simply chops off the decimal part entirely.

```
int num1 = 1, num2 = 2;
double average = (num1 + num2) / 2; // Result is 1.0 (It calculates 1, then saves it as a double)
```

- **The Fix:** Force floating-point division by making at least one of the numbers a decimal.

```
double average = (num1 + num2) / 2.0; // Result is correctly 1.5
```

Common Pitfall 1: Redundant Input Objects

A common rookie mistake is creating a brand new Scanner object every single time you want to read an input from the user.

BAD CODE (Inefficient):

```
Scanner input1 = new Scanner(System.in); // First Scanner created
System.out.print("Enter an integer: ");
int v1 = input1.nextInt();

Scanner input2 = new Scanner(System.in); // Second Scanner created (Unnecessary!)
System.out.print("Enter a double: ");
double v2 = input2.nextDouble();
```

GOOD CODE (Efficient):

You only need to create the Scanner object once at the top of your main method. You can reuse that exact same object to read as many inputs as you want.

```
Scanner input = new Scanner(System.in); // Created once!

System.out.print("Enter an integer: ");
int v1 = input.nextInt(); // Used here

System.out.print("Enter a double: ");
double v2 = input.nextDouble(); // Reused here!
```